What Is Java?

Java is a high-level, object-oriented programming language designed to be **portable, secure, and platform-independent**. Its core philosophy is:

> **"Write Once, Run Anywhere" (WORA)**

This means Java programs are compiled into **bytecode**, which runs on the **Java Virtual Machine (JVM)** rather than directly on a specific operating system.

History of Java

- Java was created in **1991** by James Gosling and his team at Sun Microsystems.
- The project was originally called **Green Project**.
- It was first designed for **interactive television systems**.
- The language was initially named **Oak** (after a tree outside Gosling's office).
- In 1995, it was renamed **Java**, inspired by Indonesian coffee.

In **1995**, Sun officially released Java to the public.

The current version of java is 25(LTS which we will using for our learing).

Long term support

Every 6 months now a new java version is released. Each version contains some new features with it, but every version is not that much mature and stable, some bugs and security issues are there.

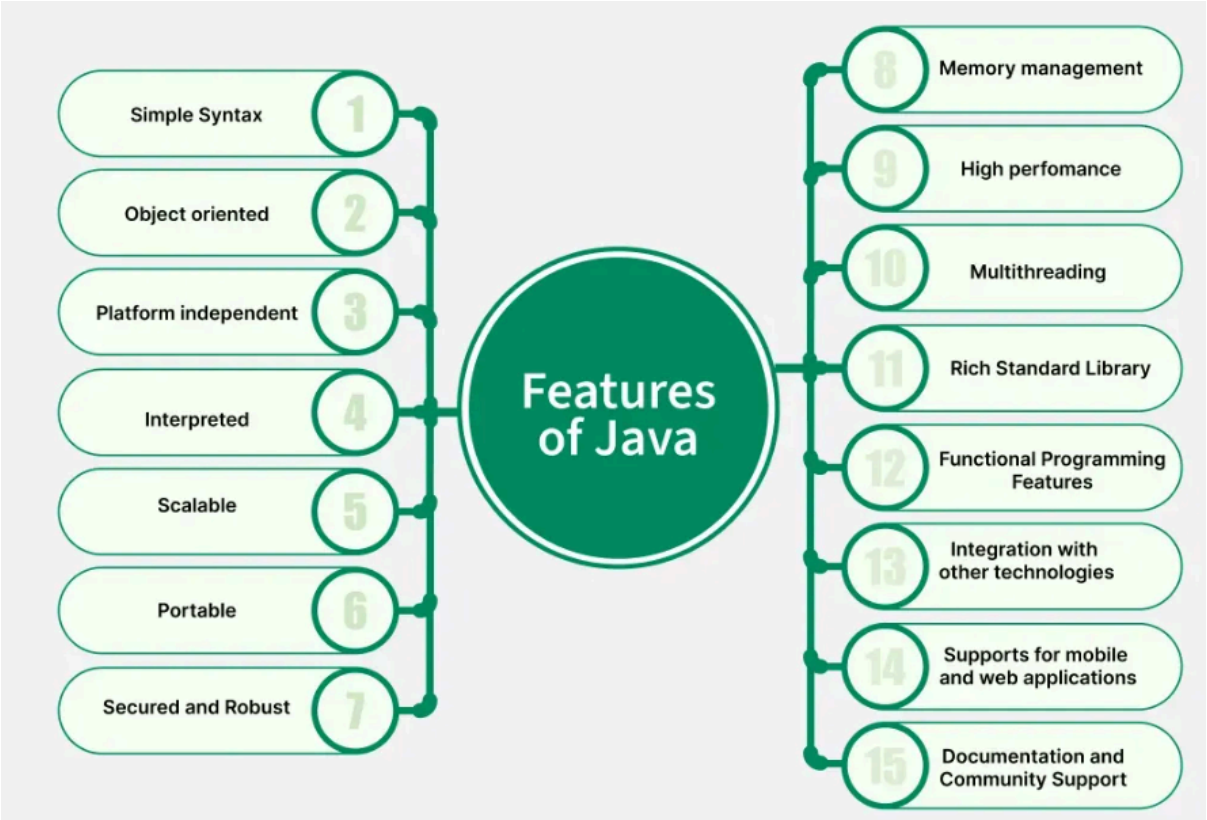
Once the version is properly tested and becomes mature java releases it as a Long term support version.

For this LTS Java will provide long term support even if there are new versions available in the market.

Always try to add a LTS version in your enterprise project so that the project will last long without any need of upgrade/migration to newer version java technology.

However if you want to explore new features launched by java you can explore it locally.

JAVA Features



1. Simple Syntax

Java syntax is very straightforward and very easy to learn. Java removes complex features like pointers and multiple inheritance, which makes it a good choice for beginners.

Example: Basic Java Program

```
class HelloWorld{
   public static void main(String[] args)
   {
```

```
        System.out.println("Hello from Java!");
    }
}
```

Output

```
Hello from Java!
```

Explanation: In Java, the execution starts with the main method, which is the entry point of any Java application. The System.out.println statement prints "`Hello from Java!`".

2. Object Oriented

Java is a pure object-oriented language. It supports core OOP concepts like,

- Class
- Objects
- Inheritance
- Encapsulation
- Abstraction
- Polymorphism

Example: The below Java program demonstrates the basic concepts of OOPs.

```
// Java program to demonstrate the basic concepts of oops
// like class, object, Constructor and method
import java.io.*;

class Student {
    int age;
    String name;
```

```
    public Student(int age, String name)
    {
        this.age = age;
        this.name = name;
    }
     // This method display the details of the student
    void display()
    {
        System.out.println("Name is: " + name);
        System.out.println("Age is: " + age);
    }
}

class Test{
    public static void main(String[] args)
    {
        Student student = new Student(30, "Ayush");
        student.display();
    }
}
```

Output

```
Name is: Ayush

Age is:
```

Explanation: In the above example, we have created a Student class and inside the class we have declared two variables age and name. A constructor is used to initialize these variables when an object of the Student class is created. In the main method we are creating an object of the student class and then we are calling the display method which is printing the name and age on the console.

3. Platform Independent

Java is **platform-independent** because of **Java Virtual Machine (JVM).**

- When we write Java code, it is first compiled by the compiler and then converted into bytecode (which is platform-independent).
- This byte code can run on any platform which has JVM installed.

4. Interpreted

Java code is not directly executed by the computer. It is first compiled into bytecode. This byte code is then understood by the JVM. This enables Java to run on any platform without rewriting code.

5. Scalable

Java can handle both small and large-scale applications. Java provides features like multithreading and distributed computing that allows developers to manage loads more easily.

6. Portable

When we write a Java program, the code first gets converted into bytecode and this bytecode does not depend on any operating system or any specific computer. We can simply execute this bytecode on any platform with the help of JVM. Since JVMs are available on most devices and that's why we can run the same Java program on different platform

7. Secured and Robust

Java is a reliable programming language because it can catch mistakes early while writing the code and also keeps checking for errors when the

program is running. It also has a feature called exception handling that helps deal with unexpected problems smoothly.

8. Memory Management

Memory management in Java is automatically handled by the **Java Virtual Machine (JVM)**.

- Java **garbage collector** reclaim memory from objects that are no longer needed.
- Memory for objects are allocated in the heap
- Method calls and local variables are stored in the stack.

9. High Performance

Java is faster than old interpreted languages. Java program is first converted into bytecode which is faster than interpreted code. It is slower than fully compiled languages like C or C++ because of the interpretation and JIT compilation process. Java performance is improved with the help of Just-In-Time (JIT) compilation, which makes it faster than many interpreted languages but not as fast as fully compiled languages.

10. Multithreading

Multithreading in Java allows multiple threads to run at the same time.

- It improves CPU utilization and enhances performance in applications that require concurrent task execution.
- Multithreading is especially important for interactive and high-performance applications, such as games and real-time systems.

- Java provides build in support for managing multiple threads. A **thread** is known as the smallest unit of execution within a process.

Example: Basic Multithreading in Java

```
// Java program to demonstrate multithreading
class MyThread extends Thread {
    public void run() {

        System.out.println("Thread is running...");
    }
}

public class Test{
    public static void main(String[] args) {
        MyThread thread = new MyThread();

        // Starts the thread
        thread.start();
    }
}
```

Output

```
Thread is running...
```

Explanation: The MyThread class extends the Thread class and overrides the run method. In the main method, an object of MyThread is created, and the start method is called to begin the execution of the thread. The run method is executed in a separate thread, printing "Thread is running..." to the console.

11. Rich Standard Library

Java provides various pre-built tools and libraries which is known as Java API. Java API is used to cover tasks like file handling, networking, database connectivity (JDBC), security, etc. With the help of these libraries developers save a lot of time and are ready to use solutions and can also build a powerful application.

12. Functional Programming Features

Since Java 8, the language has introduced functional programming features such as:

- lambda expression let us to write small block of code in a very easy way without creating full methods.
- Stream API allows data to be processed easily, Instead of writing long loops we can just filter, change, or combine data in a few lines.
- Functional interfaces are interface that contains only one method. They work perfectly with lambda expressions and help us write flexible and reusable code.

Example:

```
// Java program demonstrating lambda expressions
interface Lambda {
    int operate(int a, int b);
}

public class Test{
    public static void main(String[] args) {

        // Lambda expression
        Lambda add = (a, b) -> a + b;
        System.out.println("Addition: " + add.operate(2, 3));
    }
}
```

Output

```
Addition: 5
```

Explanation: A functional interface Lambda is defined with a single method operation. A lambda expression (a, b) -> a + b is used to implement the operate method. The main method calls the operate method using the lambda expression and prints the result.

13. Integration with Other Technologies

Java can easily work with many languages and tools as well. For example, Java can connect with C and C++ with the help of **Java Native Interface (JNI)**. Java is very popular for building **websites and webservices** like RESTful & SOAP. In Java we can use **JDBC for database connectivity** also Java is the main language for android development. As we can see Java works so well with so many different technologies that's the reason developers prefer Java more to create scalable and powerful applications.

14. Support for Mobile and Web Application

Java offers support for both web and mobile applications.

- For **web development:** Java offers technologies like JSP and Servlets, along with frameworks like Spring and Springboot, which makes it easier to build web applications.
- For **mobile development:** Java is the main language for Android app development. The Android SDK uses special version of Java and its various tools to build mobile apps for Android devices.

15. Documentation and Community Support

Java provides **documentation** which includes guides, API references, and tutorials for easy learning. Java has a l**arge** and **active global community** contributing to open-source projects, and resources. This community support helps developers **solve problems** and stay updated with new advancements.